Computational Linear Algebra for Large Scale Problems
Homework 2: Spectral Clustering

Famà Daniele s345153
Perlo Alessandro s337131

A.Y. 2024/25

1 Spectral Clustering Implementation

In this section, we are going to explain how we implemented *Spectral Clustering* for our datasets, providing a step-by-step overview of the process. We will discuss the practical decisions made during the implementation. The spectral clustering algorithm consists in the following steps.

1. Computation of the Laplacian matrix of the k-NN graph, considering the top k similarities for each point.
2. Computation of the eigenvalues and eigenvectors of the Laplacian matrix and choice of the number M of eigenvalues that are close to zero, corresponding to the weakly connected components of the graph.
3. Change of reference system by the one induced by the first M eigenvectors, k-means clustering is applied in the new space.

In the following subsections, the implementation of the first two steps of the procedure are outlined in detail.

1.1 Computation of the Laplacian Matrix

To compute the Laplacian matrix L, one has to first compute the weight matrix W whose entry $W_{i,\,j}$ corresponds to a similarity measure of the i-th and the j-th point according to the formula

$$\text{similarity}(x_i,\, x_j) = \exp \frac{-\|x_i - x_j\|}{2\sigma^2}$$

where x_i and x_j are respectively the i-th and the j-th point and σ is a scalar (in the following sections, $\sigma = 1$ will be chosen). Similarity of a point with itself is set to zero by definition. Once the matrix W is available, the k-NN graph matrix, graph where all but the top k edges for each node are set to zero, needs to be computed. We provide two distinct functions to compute the Laplacian matrix that implement two distinct ways of producing the k-NN graph, each with its own perks and disadvantages.

- `compute_laplacian`, that computes the full $n \times n$ matrix W, keeps the top k nodes for each row and finally turns the k-NN graph matrix into a sparse matrix.

- `compute_laplacian_sparse`, that never allocates a $n \times n$ full matrix, defining the sparse k-NN graph matrix storing row and column index of each entry along with its value in three vectors with $k \cdot n$ elements.

The first procedure is memory-consuming but time-efficient, since it computes just half of the elements of W (viable since the similarity function is symmetric) but its memory usage grows as $O(n^2)$. The second procedure is time-consuming but memory-efficient, since its memory usage grows as $O(n \cdot k)$ but it's necessary to recompute several times the same similarity measures.

When building the k-NN graph matrix, it may happen that x_i is a neighbor of x_j but x_j is not a neighbor of x_i. Symmetry is a desired property for the Laplacian, so we choose to add x_j as a neighbor of x_i when x_i is a neighbor of x_j but the opposite is not true. Doing so is less costly in `compute_laplacian` than in `compute_laplacian_sparse` since the latter makes use of inefficient sparse matrix assignment by index (this is a possible improvement that could be made to our code).

Finally, the matrix D is built as the diagonal matrix whose entries are the sum of elements in the corresponding rows or columns and the Laplacian is defined as $D - W$. In the cases that will be examined further, it's indifferent to choose one implementation or the other since datasets don't consist in a large number of points. For larger datasets, using `compute_laplacian_sparse` is the only viable alternative.

Notice that both `compute_laplacian` and `compute_laplacian_sparse` also return the normalized symmetric laplacian $L_{sym} = D^{-1/2} L D^{-1/2}$. The computation performed on L to implement spectral clustering are also performed on L_{sym} running the script `main_lsym`. We found no significant differences when using L_{sym} instead of L, probably because degrees across clusters are somewhat balanced, so we don't showcase results obtained using L_{sym}.

1.2 Eigenvalues computation

To compute the M smallest eigenvalues, we decided to implement the *Inverse Power Method* and the *Deflation Method*.

1.2.1 Inverse Power Method Idea

In the Inverse Power Method we apply the Power Method to the inverse of a matrix.

Let $A \in \mathbb{R}^{n,n}$ invertible, and $\lambda_1 > \lambda_2 > \cdots > \lambda_n$ its eigenvalues, then the eigenvalues of A^{-1} are: $1/\lambda_n > 1/\lambda_{n-1} > \cdots > 1/\lambda_1$. Applying the power method to A^{-1}, we would obtain its largest eigenvalues which are the reciprocal of the smallest eigenvalues of A.

1.2.2 Deflation Method Idea

Let $A \in \mathbb{R}^{n,n}$ and (x, λ) an eigenpair of A. Then, we can compute $P_1 \in \mathbb{R}^{n,n}$ orthogonal such that:

$$P_1 A P_1^T = \left(\begin{array}{c|c} \lambda & b_1^T \\ \hline 0 \\ \vdots & A_2 \\ 0 \end{array}\right)$$

We can prove that the eigenvalues of A_2 are the others $n-1$ eigenvalues of A. Iteratively, we apply to $A_k \in \mathbb{R}^{n-k+1,n-k+1}$ the Inverse Power Method and we compute the k smallest eigenvalue.

1.2.3 Methods implementation

The implementation consists in two main functions: the `deflation_eigenvalues_method` for computing multiple eigenvalues, and the `inverse_power_method` for finding a single eigenvalue-eigenvector pair.

- `inverse_power_method`

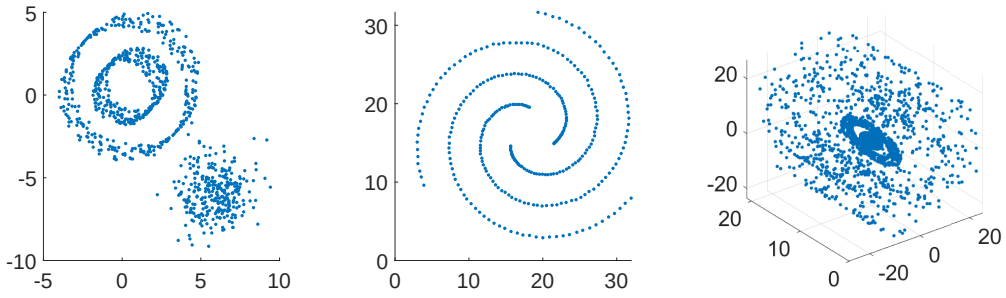
  1. input: $A \in \mathbb{R}^{n,n}$ invertible, number of max iterations, relative tolerance to check the convergence
  2. iteration:
     - computing $\tilde{v}^{(k+1)}$ solving the linear system $A\tilde{v}^{(k+1)} = v^{(k)}$
     - an estimate of the eigenvalue is computed from the Rayleigh quotient using v_k
     - update $v^{(k+1)} \leftarrow \tilde{v}^{(k+1)}$ normalized, $k \leftarrow k + 1$
  3. output: the reciprocal of the estimated eigenvalue and the eigenvector normalized

  In order to speed up the method, since a linear system where A as coefficient matrix needs to be solved at each iteration, we use the LU factorization. This results in an improvement in compute time since after having computed the LU factorization with cost $O(n^3)$, at each iteration we solve two triangular systems through back-substitution with cost $O(n^2)$. Otherwise, one would need to solve the linear system at each iteration using Gaussian Elimination, whose cost is $O(n^3)$. We chose a factorization-based approach to improve the method since the datasets we deal with are not so large. If datasets were larger and a sparse memorization format was necessary, it would be needed to use other approaches that try to improve the structure of the matrix.

- `deflation_method`

  1. input: $A \in \mathbb{R}^{n,n}$, number of max iterations
  2. initialization: initial guess of the eigenpair
  3. iteration:
     - the leading eigenvalue is computed using the inverse_power_method function.
     - A reflection transformation matrix, based on Householder transformations, is constructed using the eigenvector from the previous step.
     - I apply the householder reflection and repeat the process on the sub matrix
  4. output: the eigenvalues are stored as a diagonal matrix D, the eigenvectors are omitted in this implementation

Notice that we resort to inverse power method with deflation just to compute eigenvalues: in our application we expect many eigenvalues to be identical and equal to zero, namely eigenvalues corresponding to connected components. Then deflation can't be used to compute eigenvectors, since eigenvalues with algebraic multiplicity greater than one exist. Thus in the following sections we compare the eigenvalues computed using inverse power method with eigenvalues provided by MATLAB's `eigs`, but we use only the output of `eigs` to perform spectral clustering.



(a) Circle data set. (b) Spiral data set. (c) Synthetic data set.

Figure 1: Scatter plot of the data sets.

1.2.4 Computational cost

In the inverse power method, we compute for $k = \text{maxIter}$ iteration $\tilde{v}^{(k+1)} = A^{-1}v^{(k)}$ solving the linear system $A\tilde{v}^{(k+1)} = v^{(k)}$, so the computational cost of the so implemented method is $O(kn^3)$. The computational cost of the deflation method is equal to $O(mkn^3)$ where m is the number of eigenvalues we want to compute and $O(kn^3)$ the computational cost of the Inverse Power Method.

2 Clustering Results

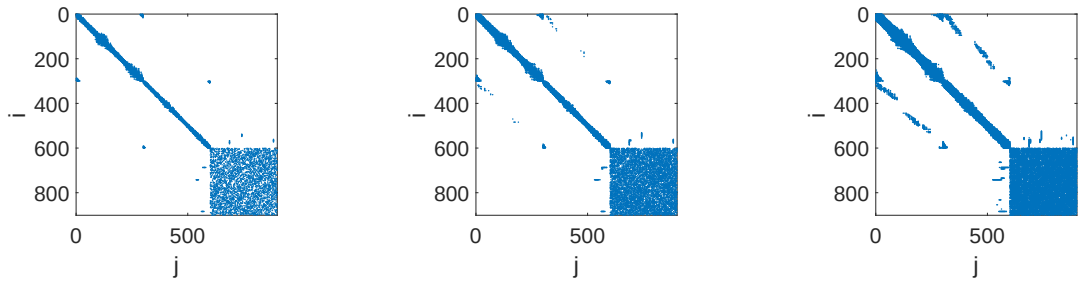
In this section, we describe the results obtained by analyzing three different types of datasets: the Circle dataset, the Spiral dataset and a 3-dimensional synthetic dataset. TODO: [Make comparison with L_{sym}]

2.1 Circle Dataset

First, we present the scatter plot of the Circle dataset. In figure 1a, two circles can be observed, one nested inside the other, along with additional points that are uncorrelated with the two circles. This dataset provides a clear example of a non-convex structure that poses challenges for clustering algorithms.

Figure 2 shows the adjacency matrix of the dataset. It is interesting to observe three main diagonal sections in the plot: the first two correspond to the two circles. As the value of k increases, the connections between the two circles also become stronger. Furthermore, analyzing the figure, we observe that many points are located near the diagonal. This indicates that a generic node i is strongly connected to its neighboring nodes. Conversely, some scattered points lie outside these diagonal blocks, indicating weak connections between node i and nodes with distant indices. Finally, we can imagine that the internal circle corresponds to the first diagonal block since it is appear to not have any connection with the third set of points, not even as k increases (figure 2c).

Both the rank of the matrix and the number of graphs derived from the *conncomp* function confirm the expected number of connected components as observed in the figures. Specifically, the results indicate that there is a single connected graph corresponding to each circle. This consistency



(a) k = 10 (b) k = 20 (c) k = 40

Figure 2: Adjacency matrix plot of spiral dataset with different values of k

with the theoretical expectations, underscores the correctness of the matrix's representation and the *conncomp* function's output in capturing the structure of the connected components.

Moreover, computing the eigenvalues, we observe two null eigenvalues and a third eigenvalue that is very small. This indicates the presence of two distinct connected components as we just have said, and the possibility of a third sub-graph with a very weak connection to the other two. In fact, figure 4b shows that the two circles belong to two different clusters and the other points on the bottom right to the third cluster.

Finally, figure 3 shows the magnitude of the first twenty smallest eigenvalue. Two eigenvalues are zero, while the third is very small. Then, we can decide to use three different clusters where two have no connection one from each other while the third one is weakly connected to the others.

Figure 4 shows some of the possible clustering solution where it has been included one of the case with $M > 3$ in which we can see that decrease the clear distinction across different cluster. The best solution is given for $M = 3$ where we can notice a cluster for each circle and a third cluster related to the "cloud" of points on the bottom right.

2.2 Spiral Dataset

The spiral dataset contains three columns: first two correspond to x-values and y-values of the points while the third one contains the index of the correct cluster.

Figure 1b shows three different spiral. We expect the clustering algorithm to produce a separate cluster for each spiral present in the dataset. This is another example where spectral clustering is among the best techniques at identifying groups of similar objects.

Figure 5 shows the distribution of non-zero values in the adjacency matrix for different k values. Although we can immediately notice the good distinction between the three spiral along the diagonal, there are also significant connections between different spirals, which increase for higher k values. This is probably caused by the low density of the spiral.

Moreover, where the density of points is higher, the corresponding diagonal sections in the adjacency matrix are larger As mentioned above, we imagine that the indices corresponding to the tail of the spiral (low density, more likely to form connections across different spirals) precede the head indices corresponding to the internal part of the spiral (where the density is higher) which show a more compact and better pattern in the adjacency matrix.

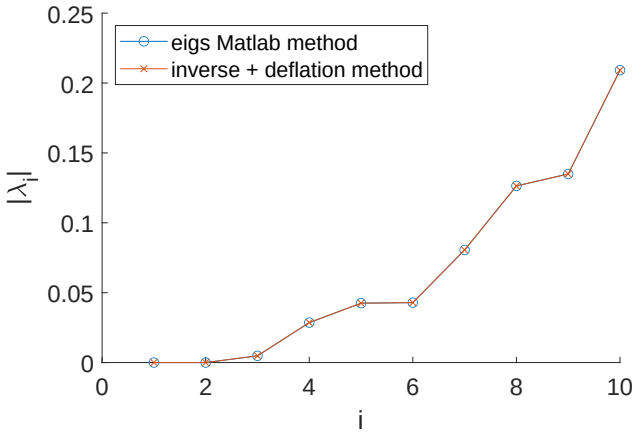
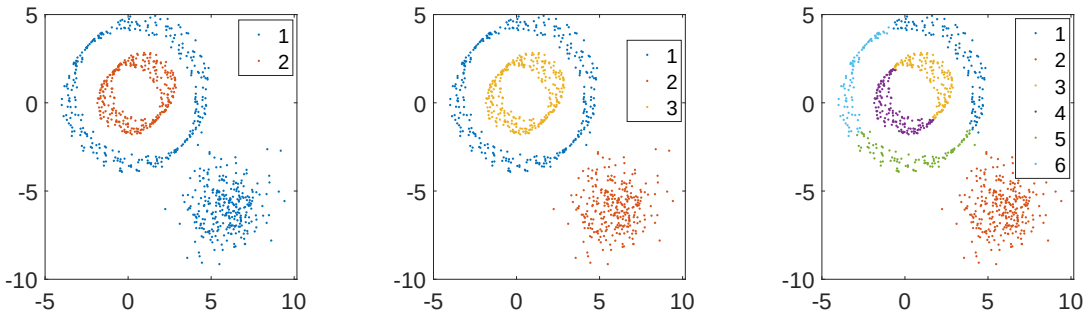
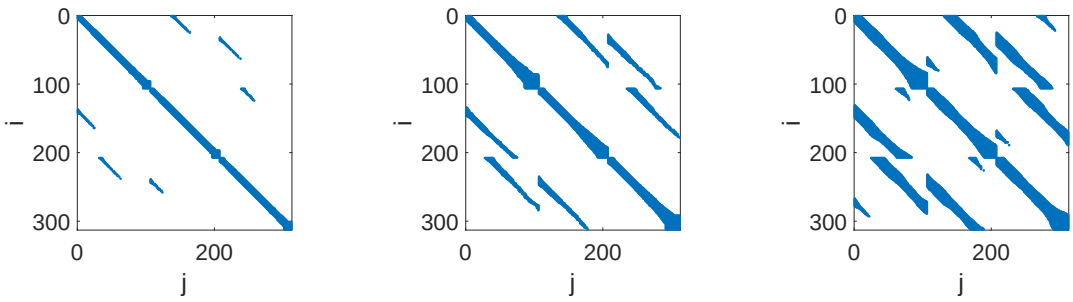


Figure 3: Eigenvalues of Laplacian matrix from Circle dataset with k = 10



(a) M = 2 (b) M = 3 (c) M = 6

Figure 4: Scatter plot of the Circle dataset with points coloured according to their clusters (among M)



(a) k = 10 (b) k = 20 (c) k = 40

Figure 5: Adjacency matrix plot of the spiral dataset for different k values.

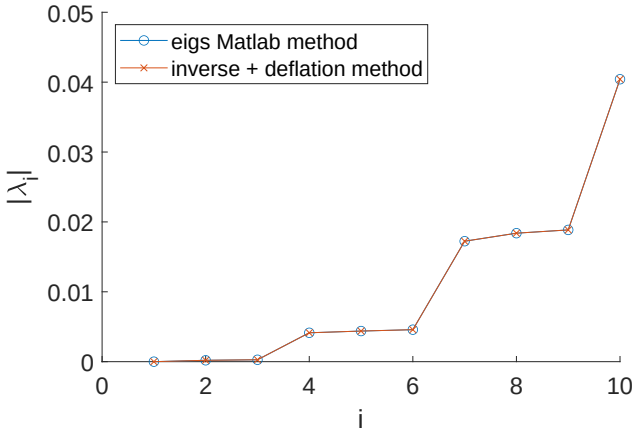
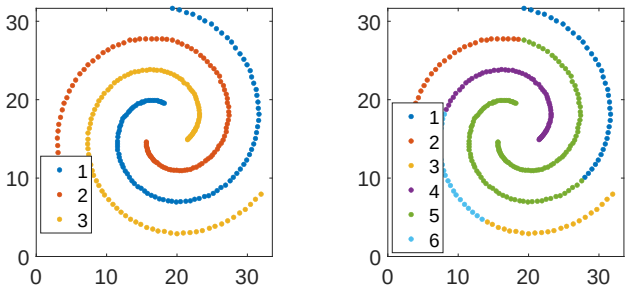


Figure 6: Eigenvalues of Laplacian matrix from Spiral dataset with k = 10



(a) M = 3 (b) M = 6

Figure 7: Scatter plot of the Spiral dataset with points colored according to their clusters.

Both the rank of the matrix and the number of graphs derived from the *conncomp* function give us a single connected component even though we expect three. Decreasing the value of k, or increasing the density of the spiral will ensure our previsions.

Although there is only one connected component, we could identify three weakly connected sub-graphs as we can figure out with figure 6. In fact there is only one vanishing eigenvalue and two very small eigenvalues indicating that we can increase the number of cluster since they would have very few similarities. Looking at the eigenvalue plot, it can be observed that, for small values of M, there are significant increases of magnitude, every three eigenvalues. This behavior is due to the symmetry of the three spirals, which leads to a uniform response of the clustering algorithm on each of the spirals as we can see in figure 7.

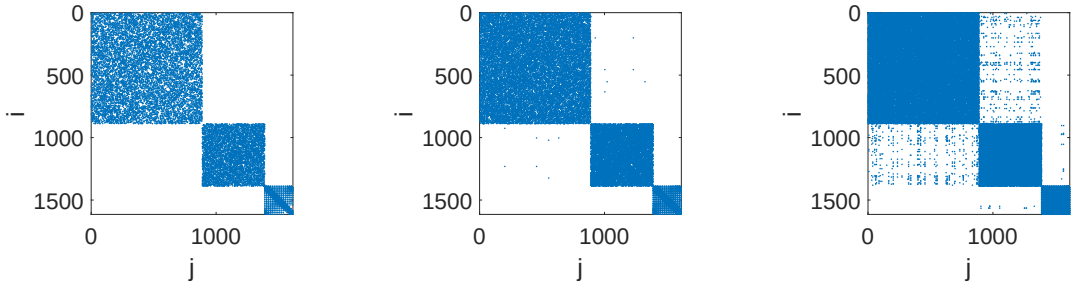
In conclusion, although there is a single connected component in the graph, using $M = 3$ allows us to identify a subgraph corresponding to each of the three spirals. Each spiral forms a subgraph that is weakly connected to other spirals, allowing the clustering algorithm to recognize it as a distinct cluster.

2.3 Synthetic 3D Dataset

The dataset shown in figure 1c to test spectral clustering on a 3D dataset and to compare it with other clustering techniques. The dataset is composed by a swiss roll whose points in the range (8, 12) along the first axis were removed so that 7 distinct clusters are produced. Nested inside the swiss roll we find a torus containing a sphere. Then the number of distinct clusters amounts to 9.

Figure 8 shows the distribution of non-zero values in the adjacency matrix of the dataset for different values of k. One can notice three main diagonal sections corresponding respectively to the swiss roll, the circle and the sphere. Residual connections between a diagonal block and the next one appear as the value of k increases (as shown in 8c): this is an expected behavior being the torus nested inside the swiss roll and the sphere nested into the torus.

Both the rank of the matrix and the `conncomp` hint that 3 connected components exist for $k = 10$. However, the plot of the moduli of the eigenvalues reveal that there are 9 weakly connected components: after the 9^{th} eigenvalue, moduli start to grow steeply. Choosing $M = 9$ leads all the clusters to be correctly identified when the spectral clustering algorithm previously discussed is applied to the dataset, as shown in figure 10b.



(a) k = 10 (b) k = 20 (c) k = 40

Figure 8: Adjacency matrix plot of the synthetic dataset for different k values.

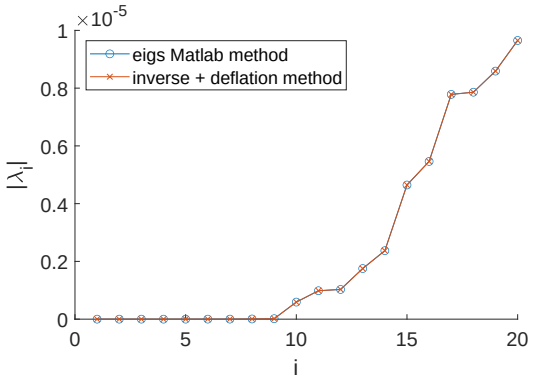
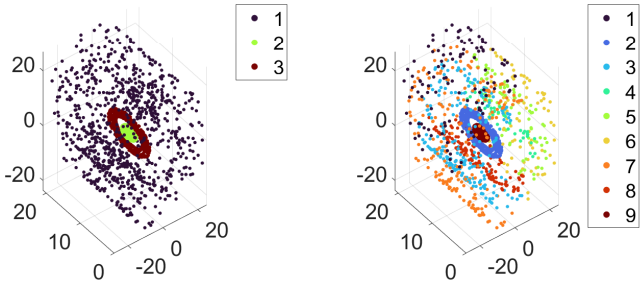
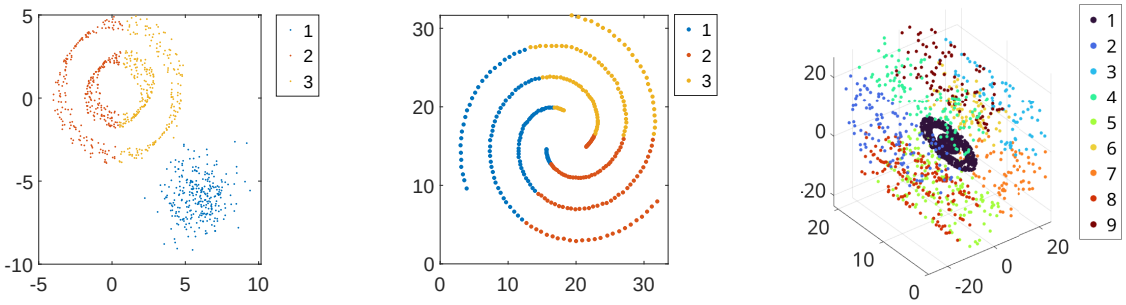


Figure 9: Eigenvalues of Laplacian matrix from Synthetic dataset with $k = 10$



(a) M = 3 (b) M = 9

Figure 10: Scatter plot of the synthetic dataset with points colored according to their clusters.



(a) Circles data set. (b) Spiral data set (c) Synthetic data set.

Figure 11: Results of the k-means clustering algorithm applied to different datasets.

3 Comparison with other clustering methods

In the following we compare the results obtained through spectral clustering with results obtained using other clustering algorithm. That is, we focus on K-means for distance-based clustering and DBScan for density-based algorithms.

3.1 Comparison with K-means clustering

The k-means clustering has been applied using as a value for the number of clusters k the number of expected clusters that is 3 for both circles and spiral dataset, 9 for the synthetic data set.

1. In the circles data set, as shown in figure 11a the convex set of points is correctly mapped to one of the centroids by the algorithm, while the two concentric circles are not.

2. In the spiral dataset, as shown in figure 11b the non-convex spirals are not correctly recognized as clusters by the algorithm.

3. In the synthetic dataset, as shown in figure 11c the results are similar as in the previous cases. Being the torus non-convex and containing a sphere, the torus and the sphere are marked as a single cluster. The Swiss roll is split into clusters in a way similar to how the spiral was split into clusters.

In general, effectiveness of k-means clustering is worse than effectiveness of spectral clustering when identifying non-convex or nested shapes, while it is comparable when clusters are convex.

3.2 Comparison with density based methods

DBScan has been applied using as value for the size of the neighborhood ε a value chosen looking at the k-Nearest Neighbors distance graph with $k = 4$ in figure 12. The distance graph is built considering the mean distance from the k nearest neighbors for each point, then the distances are plotted sorted in increasing order. Adopting the elbow method, ε has been chosen as the value where there is an elbow (i.e., steep change in slope) in the curve. The values of ε are set to 0.5 and

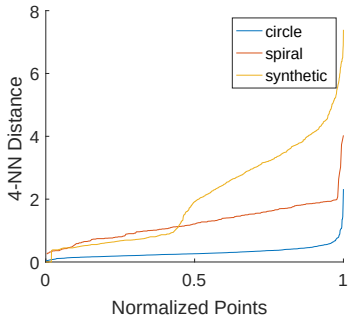
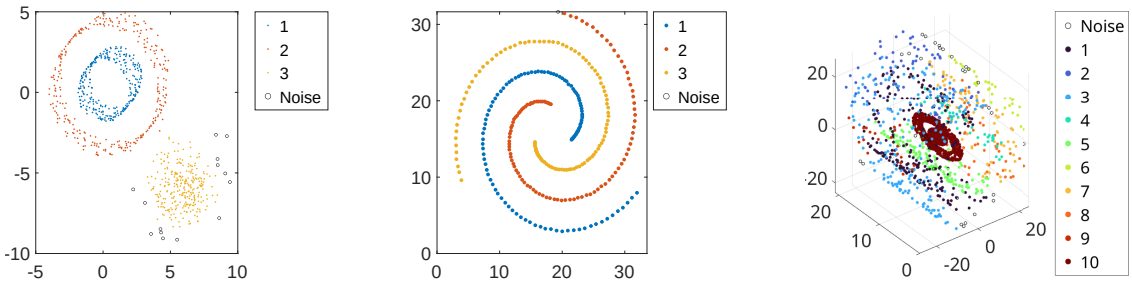


Figure 12: k-NN distance graph for the given datasets, the number of points is normalized to 1 so that all curves span the same range on the horizontal axis.



(a) Circles data set. (b) Spiral data set (c) Synthetic data set.

Figure 13: Results of the DBScan clustering algorithm applied to different datasets.

2 respectively for the circle data set and the spiral data set. For the synthetic dataset, applying this rule is slightly more complex, since there are 2 steep changes in slope, namely 1 and 4.5. These distinct elbows arise from the fact that the mean distances of points in the sphere and the torus are much smaller than the mean distances of points in the Swiss roll. It has been chosen $\varepsilon = 4.5$ since choosing the other value, all points in the Swiss roll are classified as noise, even though the torus and the sphere are correctly separated.

- In the circle dataset, as shown in figure 13a points are correctly clustered.
- In the spiral dataset, as shown in figure 13b points are correctly clustered but a point of a spiral is classified as noise: this result could improve varying the minimum number of points hyperparameter of DBScan.
- In the synthetic dataset, as shown in figure 13c, the torus and the sphere are grouped in a single cluster: this is due to the value of ε as previously discussed. The slices of the Swiss roll are almost correctly clustered, but some points along the cut in the roll are classified as noise, and some less dense portions of the roll are split into a higher number of clusters than necessary.

In general, effectiveness of DBScan is comparable to effectiveness of spectral clustering when density of points is uniform across clusters, while it is worse when clustering groups of points with non-uniform densities. However, the gap is not so meaningful and one could opt for DBScan in a nonuniform density scenario if interested in outlier identification.